Деталь — Поставщик

Доронкин Артём Сергеевич

Группа ИУ5-35Б

27 октября 2026 года

Язык реализации: Python 3.

Постановка задачи

Г1. Вывести поставщиков на «А» и их детали (один-ко-многим).
Г2. Найти максимальную цену детали у каждого поставщика
и отсортировать поставщиков по этой цене по убыванию.
Г3. Вывести связанные пары «деталь — поставщик»
(многие-ко-многим), отсортированные по поставщикам.

Модель данных
Part: ID, название, цена в рублях, ID основного поставщика.
Supplier: ID и название поставщика.
PartSupplier: ID детали и ID поставщика для связи M:N.

Данные: 3 поставщика, 5 деталей, 5 связей M:N.
Связи 1:N и M:N задаются независимо, соединение — по ID.
Цена является свойством детали. Поставщики без деталей
не включаются в Г2, поскольку максимум не определён.

Условия: ugapanyuk/courses_current на GitHub,
python/rk/pcpl/pcpl_rk_1.pdf (стр. 1, 3–4).

Текст программы: классы и данные

```
"""РК-1, вариант Г-20. Доронкин Артём Сергеевич, ИУ5-35Б."""
from operator import itemgetter


class Part:
    """Деталь: идентификатор, название, цена в рублях, поставщик."""
    def __init__(self, id, name, price, supplier_id):
        self.id = id
        self.name = name
        self.price = price
        self.supplier_id = supplier_id


class Supplier:
    """Поставщик."""
    def __init__(self, id, name):
        self.id = id
        self.name = name


class PartSupplier:
    """Связь детали и поставщика многие-ко-многим."""
    def __init__(self, part_id, supplier_id):
        self.part_id = part_id
        self.supplier_id = supplier_id


suppliers = [
    Supplier(1, 'Альфа'),
    Supplier(2, 'Вектор'),
    Supplier(3, 'Атлас'),
]

parts = [
    Part(1, 'Болт', 20, 1),
    Part(2, 'Гайка', 10, 1),
    Part(3, 'Подшипник', 500, 2),
    Part(4, 'Шестерня', 800, 2),
    Part(5, 'Вал', 1200, 3),
]

# Болт поставляют Альфа и Атлас; Альфа поставляет Болт и Гайку.
parts_suppliers = [
    PartSupplier(1, 1),
    PartSupplier(2, 1),
    PartSupplier(3, 2),
    PartSupplier(5, 3),
    PartSupplier(1, 3),
]
```

Текст программы: запросы и запуск

```
def query_g1(parts, suppliers):
    """Поставщики на А и их детали (один-ко-многим)."""
    return [
        (s.name, [p.name for p in parts if p.supplier_id == s.id])
        for s in suppliers if s.name.startswith('A')
    ]


def query_g2(parts, suppliers):
    """Максимальные цены по поставщикам, по убыванию."""
    maxima = [
        (s.name, max(p.price for p in parts if p.supplier_id == s.id))
        for s in suppliers
        if any(p.supplier_id == s.id for p in parts)
    ]
    return sorted(maxima, key=itemgetter(1), reverse=True)


def query_g3(parts, suppliers, links):
    """Все связанные пары многие-ко-многим, по поставщику."""
    pairs = [
        (p.name, s.name)
        for link in links
        for p in parts if p.id == link.part_id
        for s in suppliers if s.id == link.supplier_id
    ]
    return sorted(pairs, key=itemgetter(1))


def main():
    print('Г1. Поставщики на А и их детали (один-ко-многим):')
    for supplier, names in query_g1(parts, suppliers):
        print(f'{supplier}: {", ".join(names)}')

    print('\nГ2. Максимальная цена детали (по убыванию):')
    for supplier, price in query_g2(parts, suppliers):
        print(f'{supplier}: {price} руб.')

    print('\nГ3. Деталь - поставщик (многие-ко-многим):')
    for part, supplier in query_g3(parts, suppliers, parts_suppliers):
        print(f'{part} - {supplier}')


if __name__ == '__main__':
    main()
```

Результаты выполнения

Команда: python3 main.py

```
Г1. Поставщики на А и их детали (один-ко-многим):
Альфа: Болт, Гайка
Атлас: Вал

Г2. Максимальная цена детали (по убыванию):
Атлас: 1200 руб.
Вектор: 800 руб.
Альфа: 20 руб.

Г3. Деталь - поставщик (многие-ко-многим):
Болт - Альфа
Гайка - Альфа
Вал - Атлас
Болт - Атлас
Подшипник - Вектор
```

Проверка результата

Г1: Альфа поставляет Болт и Гайку, Атлас — Вал.
Г2: максимумы равны 1200, 800 и 20 руб.; порядок убывающий.
Г3: Болт присутствует у двух поставщиков, что демонстрирует связь многие-ко-многим. Пары упорядочены по поставщикам.

Команда проверки: python3 -m unittest -v
Результат: 6 тестов, OK.
Проверены все запросы, пустые данные, пустой поставщик и соединение по ID при одинаковых названиях поставщиков.

Вывод: реализованы три запроса варианта Г-20 с использованием comprehensions, генераторов, max и sorted.